

Microservice Architecture

1. What Is Microservice Architecture?

Microservice Architecture is an architectural style where a large application is built as a **collection of small, independent, and loosely coupled services**.

Each service:

- Represents **one business capability**
- Has its **own database** (most commonly)
- Runs **independently**
- Can be **deployed, scaled, and maintained** without impacting others
- Communicates with other services via **APIs** (REST, gRPC, messaging, etc.)

Analogy

Think of an e-commerce system:

- Product Service
- Order Service
- Payment Service
- Inventory Service
- User/Identity Service

Each works independently like specialized departments in a company.

2. Why Microservices? (Problem with Monolithic Architecture)

Monolith Architecture Characteristics

- All features in **one single application**
- One **shared database**
- Deployment as a **single unit**

Problems of Monoliths

Problem	Explanation
Hard to scale	You scale the whole app even if one module needs scaling
Slow deployments	Small change requires redeploying entire application
Hard to maintain	Code grows messy; dependencies get tangled
Low agility	Teams can't work independently
Technology lock-in	Entire system often uses one tech stack

Microservices Solve These:

- **Independent scaling, development, and deployment**
- **Fault isolation** (one service failure doesn't crash the whole system)
- **Polyglot freedom** (different languages, databases if needed)
- **Better CI/CD support**
- **High agility for teams**

3. Characteristics of a Microservice

1. Single Responsibility Principle (SRP)

One microservice = one business capability

Example: Payment Service handles only payments, not order validation.

2. Own Database (Decentralized Data Management)

No shared DB.

Order Service should not directly read Product DB.

Instead, they communicate via APIs or events.

3. Lightweight Communication

- HTTP/REST
- gRPC(Google Remote Procedure Call)
- Message brokers (RabbitMQ, Kafka, Azure Service Bus)

4. Independent Deployment

Update Product Service anytime without redeploying other services.

5. Loose Coupling and High Cohesion

Internal logic tightly grouped

Interaction with other services minimal and via well-defined contracts.

4. How Microservices Communicate

A. Synchronous Communication (Request-Response)

Common for direct queries.

Examples:

- REST APIs
- gRPC

Pros: Simple

Cons: Tight coupling, cascading failures possible

B. Asynchronous Communication (Event-Based)

Used for long-running processes.

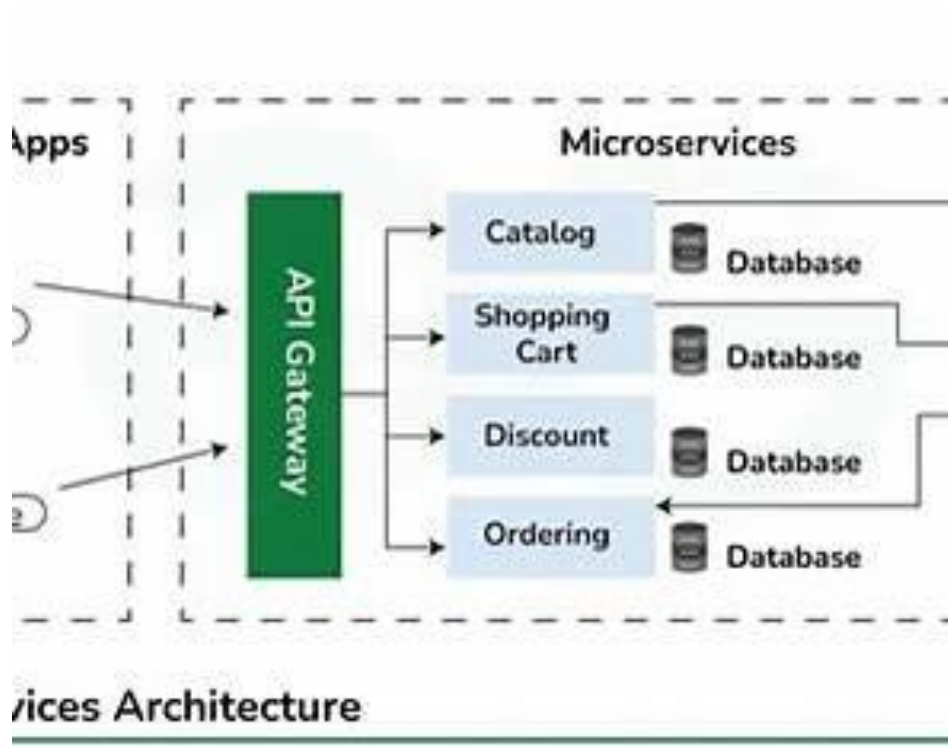
Example:

- OrderPlaced event published by Order Service
- Inventory Service listens and updates stock
- Payment Service triggers payment workflow

Pros: Loose coupling, reliability, scalability

Cons: System becomes eventually consistent

5. API Gateway – The Entry Point



Clients never call microservices directly.

They call an **API Gateway**, which:

- Routes traffic to services
- Handles authentication (JWT)
- Performs rate limiting
- Aggregates responses
- Provides a single entry URL

Common gateways:

- Ocelot (for .NET)
- YARP
- Kong
- NGINX

6. Database Pattern in Microservices

1. Database Per Service (Recommended)

Each service has its own DB. Prevents coupling.

Example:

- ProductService — SQL Server
- OrderService — PostgreSQL
- IdentityService — MySQL

2. Shared Database (NOT Recommended)

Because it couples services tightly and breaks independence.

7. Service Discovery

In distributed systems, services scale dynamically. Each instance has different IPs.

Service Discovery helps clients locate services.

Examples:

- Eureka (Netflix OSS)
 - Consul
 - Kubernetes Service Discovery
-

8. Microservices Deployment

Containerization

Microservices are most commonly deployed as:

- **Docker containers**
- Managed by **Kubernetes (K8s)**

Benefits:

- Portability

- Auto-scaling
- Rolling upgrades
- Load balancing

9. Challenges in Microservices (Important)

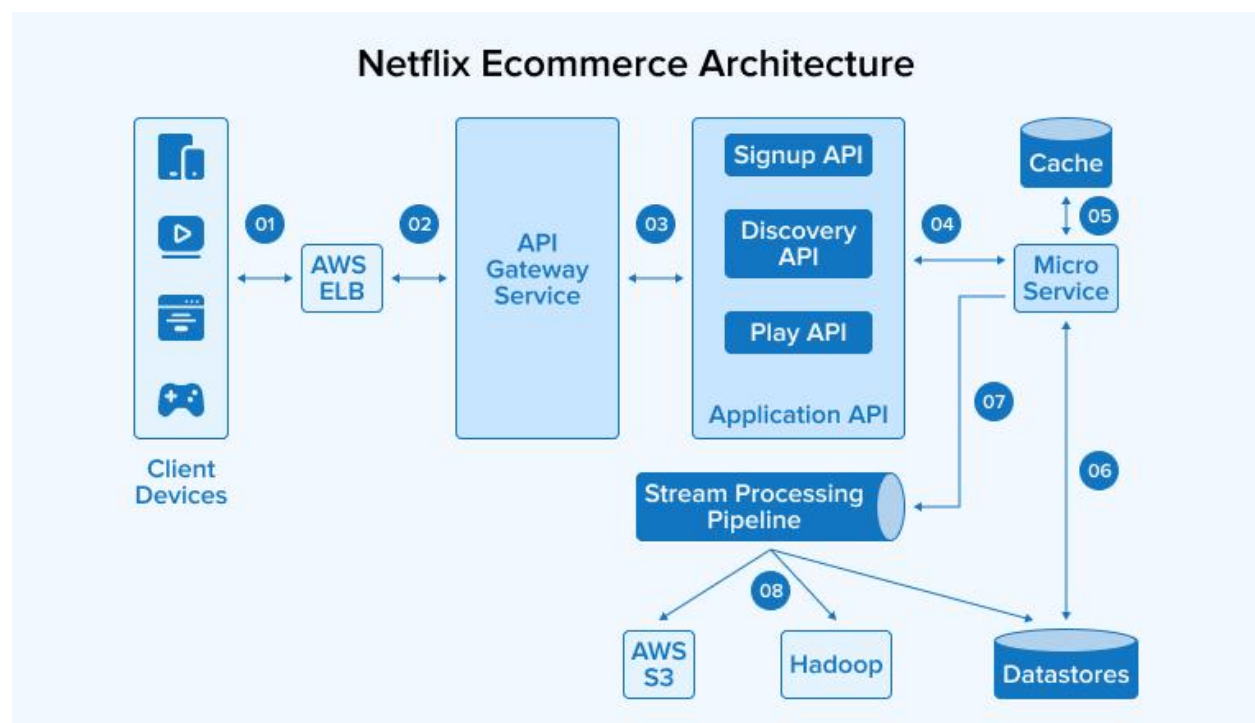
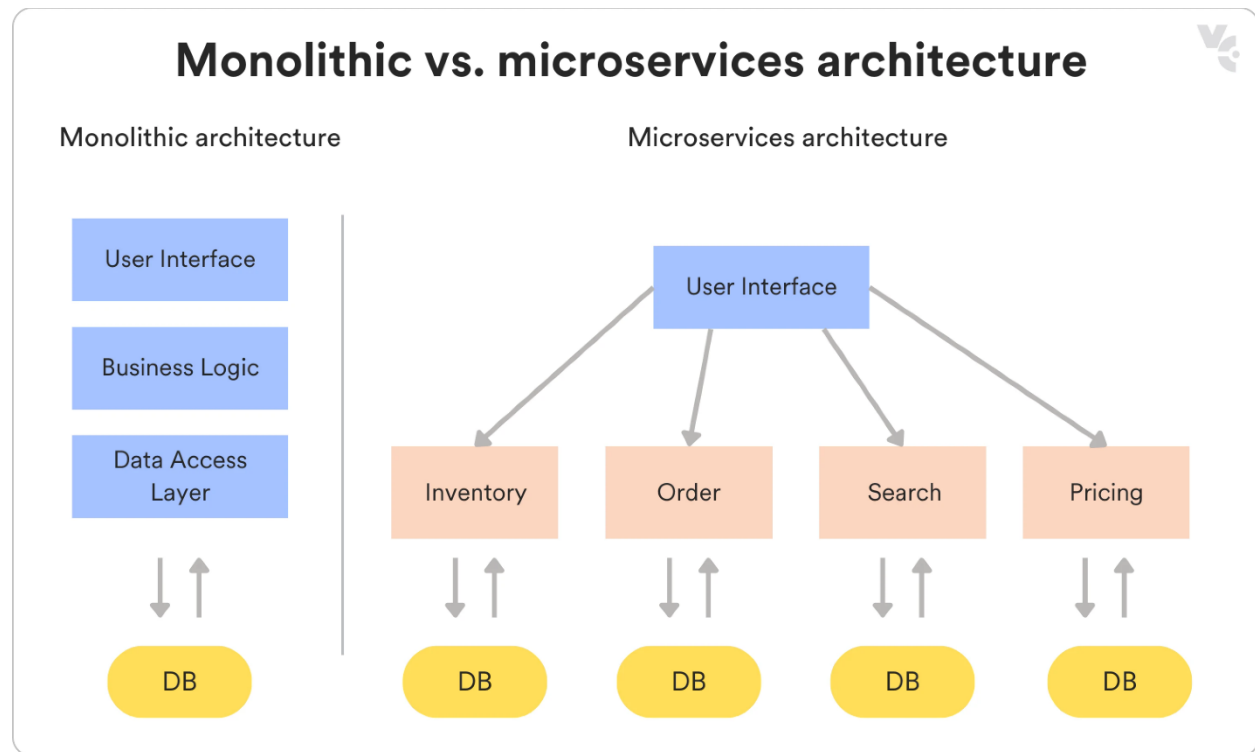
Microservices solve many problems but introduce new ones:

Challenge	Description
Distributed data	No joins across DBs, event-driven consistency
Network latency	Multiple service calls → slower response
Debugging difficulty	Logs scattered across multiple services
DevOps complexity	Requires CI/CD, Docker, Kubernetes
Monitoring	Must track service health, uptime, logs

Tools:

- **Logging:** ELK, Serilog + Seq
 - **Tracing:** OpenTelemetry, Jaeger
 - **Metrics:** Prometheus, Grafana
-

10. Real Example: E-Commerce Microservices Architecture



Microservice	Responsibility	Database
Product Service	Product CRUD, catalog	SQL Server
Order Service	Creates orders, validation	PostgreSQL
Payment Service	Payment processing	External APIs
Inventory Service	Stock updates	MongoDB
Identity Service	Authentication / JWT	SQL Server

Flow:

1. Customer places order
2. Order Service creates entry
3. Order Service publishes **OrderCreated event**
4. Inventory Service reduces stock
5. Payment Service charges user
6. Order Service updates order status to Completed

This is a distributed, event-driven microservices system.

11. When Should You Use Microservices?

Choose Microservices When:

- Application is **big and complex**
- You need **independent scaling**
- You want **independent team ownership**
- You plan for **continuous deployment**
- You have **DevOps maturity**

Avoid Microservices When:

- The team is small

- Application is simple
 - No DevOps/dev experience
 - You cannot handle distributed system complexity
-

12. Summary

Microservices provide:

- Agility
- Independently scalable and deployable units
- Better fault isolation
- Technology freedom

But they require:

- Strong DevOps
- Logging/monitoring
- Event-driven patterns
- Team maturity